

Основы **Python** с нуля – шпаргалка

Правило → пример → результат · переменные, типы, операции, константы, функции и аргументы · Python 3

Как пользоваться шпаргалкой. Каждый урок устроен одинаково: сначала правило, затем пример кода и результат. Прочитайте правило, закройте ответ в примере и попробуйте предсказать, что выведет программа, – а потом сверьтесь. Код набран моноширинным шрифтом на тёмном фоне; после знака # – комментарий с результатом. Уроки идут по порядку: присваивание → типы → операции → функции → аргументы → результат.

Часть 1. Переменные, типы, операции и константы

01 Как читать и выполнять код

Python выполняет инструкции по порядку. В учебных программах пишите одно действие на строке. Выражение вычисляет значение; инструкция выполняет действие. Вывод появляется после вызова print().

```
product = 'Малина'  # присвоить строку
quantity = 15       # присвоить число
print(product)     # Малина
print(quantity)    # 15
```

Литерал – значение прямо в коде: 15, 3.14, 'Малина'. **Имя** – обозначение связанного с ним значения: quantity. **Выражение** – например quantity + 10.

Комментарий: # вне строки начинает примечание до конца строки. Закомментировать код – добавить #; раскомментировать – убрать его. В '#' решётка является текстом.

Порядок изучения: присваивание → типы → операции → функции → аргументы → результат. Прочитайте правило, закройте ответ примера и предскажите вывод.

02 Переменные и присваивание

имя = выражение: сначала Python вычисляет правую часть, затем связывает имя слева с результатом. Знак = означает присваивание, а не проверку равенства.

```
count = 15
total = count + 10  # total: 25
count = 20          # total всё ещё 25
count = count + 1   # count: 21
```

Новое присваивание заменяет связь имени. Значение total не пересчитывается: это готовый результат.

`count = 15` → `15` `count = 20` → `20`

Имена: осмысленные английские слова, snake_case: jar_count, jar_volume. Регистр важен: count и Count – разные имена. Не начинайте имя с цифры; не используйте пробелы и дефисы.

Избегайте: непонятных a, nm, транслита banka и имён встроенных функций print, int, str, type. До чтения имени ему должно быть присвоено значение.

03 Типы данных и типизация

str – строка: 'Тоня', "текст", '31', ''. Кавычки по краям должны совпадать. **int – целое число:** 15, 0, -2. **float – число с плавающей точкой:** 3.14, 0.001; разделитель – точка.

```
print(type('31'))  # <class 'str'>
print(type(31))    # <class 'int'>
print(type(3.14))  # <class 'float'>
```

Динамическая типизация: тип принадлежит объекту; одно имя можно последовательно связать со значениями разных типов: x = 5, затем x = 'текст'.

Сильная типизация: Python не складывает несовместимые типы по догадке. Выражение 1 + '2' вызывает TypeError. Выберите: складывать числа или объединять строки.

Уточнение: совместимые числовые типы могут участвовать в общей операции: 1 + 2.5 даёт 3.5. Поэтому правило «Python никогда сам не преобразует типы» слишком категорично. Для проверки неожиданного значения временно выведите **print(x, type(x))**. Число 31 и строка '31' печатаются похоже, но ведут себя по-разному.

04 Операции с числами и строками

```
2 + 3          # 5, сложение
7 - 2          # 5, вычитание
3 * 4          # 12, умножение
7 / 2          # 3.5, деление
3 ** 2         # 9, степень
2 + 3 * 4      # 14
(2 + 3) * 4    # 20
```

В этих формулах: скобки задают группировку; степень выполняется раньше умножения и деления, затем идут сложение и вычитание. / даёт float даже при делении нацело: 4 / 2 → 2.0.

```
'перс' + 'тень' # 'перстень'
'ля' * 3        # 'ляляля'
'25' + '50'     # '2550', не 75
'А' + ' ' + 'Б' # 'А Б'
```

+ для строк – конкатенация (склеивание), *** с целым числом** – повторение. Пробелы при склеивании сами не добавляются. Вычитание строк и сложение строки с числом вызывают TypeError.

05 Преобразование типов

str(x) создаёт строковое представление; **int(x)** преобразует подходящее значение в целое число. Результат нужно сохранить или сразу использовать.

```
number = 100
text = str(number)  # '100'
print(type(number)) # <class 'int'>
print(type(text))   # <class 'str'>
print(int('25') + int('50')) # 75
```

Вызов str(number) сам по себе не меняет number. Если написать number = str(number), имя будет связано уже со строкой.

```
int('123')     # 123
int('123abc')  # ValueError
int('3.14')    # ValueError
float('3.14')  # 3.14
int(3.9)       # 3, не округление
```

Выбор преобразования: для расчётов переводите числовой текст в число; для склеивания с текстом — число в строку. float() и поведение int() с дробями здесь даны как уточнение к материалам курса.

06 Вывод через print

print(a, b, c) печатает значения разных типов, по умолчанию разделяя их пробелом и завершая вывод переводом строки. Преобразование в str() для отдельных аргументов не нужно.

```
count = 15
print('Банок:', count)
# Банок: 15
print('Банок: ' + str(count))
# Банок: 15
```

Запятая и + решают разные задачи. Запятая разделяет аргументы print(); + сначала создаёт одну строку. print('Банок: ' + count) неверно: ошибка возникает ещё до печати.

```
berries = 25
vegetables = '12'
total = berries + int(vegetables)
print('Всего заготовлено', total,
      'банок.')
# Всего заготовлено 37 банок.
```

Внутри скобок разрешены переносы строк. В тренажёре важны точные пробелы, регистр и знаки препинания.

07 Константы и формула объёма

Константа – имя для значения, которое по смыслу не должно меняться. Имена: UPPER_SNAKE_CASE. Python не запрещает менять константы.

```
PI = 3.14
CM3_TO_LITERS = 0.001
height = 9
radius = 5
volume = (PI * height * radius ** 2
          * CM3_TO_LITERS)
```

$V = \pi \times h \times r^2$

$см^3 \times 0.001 = литры$

Результат: 0.7065 л. Если h и r в сантиметрах, π × h × r² даёт см³. Один литр – 1000 см³, поэтому умножаем на 0.001. Нужен радиус, не диаметр.

Польза: смысл чисел понятен; правка в одном месте. 3.14 – приближение π.

Python 3 • Читайте блоки слева направо, затем сверху вниз • Печать A4, масштаб 100%

08 Проверка первого листа

Предскажите результат без запуска. ① '7' + '3' ② int('7') + 3 ③ 'xa' * 2 ④ type('3.14') ⑤ 2 + 3 ** 2. Затем проверьте себя ниже.

Найдите ошибку: ⑥ print('Всего: ' + 12) ⑦ int('два') ⑧ print(unknown), если unknown ранее не определено.

Ответы: ① '73' ② 10 ③ 'хаха' ④ <class 'str'> ⑤ 11. ⑥ TypeError: используйте str(12) или отдельный аргумент. ⑦ ValueError: строка не задаёт целое число. ⑧ NameError: имя неизвестно.

Чек-лист: строка в кавычках; число без кавычек; дробь через точку; имя задано до использования; типы совместимы; при + между словами есть пробелы.

Разбор ошибки: прочитайте последнюю строку сообщения, найдите указанную строку кода, проверьте значения и типы, исправьте причину и запустите заново.

Важно для новичка: комментарии с ответами не выполняются. Не переписывайте номера строк и кнопки «Запустить код» из тренажёра в программу.

📋 Часть 2. Функции, аргументы и возвращаемые значения

09 Функция и её вызов

Функция – именованный блок кода для одной задачи. Принцип **DRY** (Don't Repeat Yourself) предлагает вынести повторяющиеся действия в функцию и вызывать её с разными данными.

```
def welcome():
    print('Добро пожаловать!')
welcome()  # выполнить тело
welcome()  # выполнить ещё раз
```

Синтаксис: def, пробел, имя, круглые скобки, двоеточие. Тело пишется с отступом в 4 пробела. Возврат к прежнему отступу завершает тело. Не смешивайте табуляцию и пробелы.

Объявление не выполняет тело. Вызов – имя со скобками: welcome(). Просто welcome обозначает саму функцию. К моменту вызова определение функции должно быть выполнено.

Имена действий: calculate_volume, show_message, get_total. Используйте snake_case; не занимайте имена print и других встроенных функций.

10 Параметры и аргументы

Параметр – имя в объявлении функции. **Аргумент** – конкретное значение, переданное при вызове. Параметры получают значения аргументов и используются в теле.

```
def welcome(profession, name):
    print('Привет,', profession, name)
welcome('фермер', 'Тоня')
# Привет, фермер Тоня
```

`'фермер'` → `profession` `'Тоня'` → `name`

Аргументом может быть литерал, значение переменной, результат выражения или вызова другой функции. Имя переменной при вызове не обязано совпадать с именем параметра.

```
user_name = 'Женя'
welcome('садовод', user_name)
```

Если обязательный аргумент пропущен, Python выдаст TypeError. Лишний аргумент тоже может вызвать TypeError. Проверьте имена, порядок и число параметров.

11 Как передавать аргументы

```
def action(name, food='ягоды'):
    print(name, 'заготавливает', food)
action('Женя', 'морковь')
action('Женя', food='морковь')
action(food='морковь', name='Женя')
```

Все три вызова печатают одно и то же. **Позиционные** аргументы сопоставляются слева направо. **Именованные** задают параметр явно через имя=значение; их порядок можно менять.

```
action('Женя')  # food: 'ягоды'
action(name='Женя')  # то же самое
action('картофель')  # это имя!
```

В обычных вызовах этого курса: сначала позиционные аргументы, затем именованные. action(name='Женя', 'морковь') – SyntaxError. action('Женя', name='Тоня') – TypeError: name задан дважды. Именованный аргумент должен совпадать с именем параметра. Значение по умолчанию для этого не обязательно. У некоторых встроенных функций есть ограничения на именованные аргументы.

12 Умолчания и локальные имена

Значение по умолчанию задаётся в объявлении: def action(name, food='ягоды'). Параметр name обязателен; food можно пропустить. Переданный аргумент заменяет значение по умолчанию. В показанной форме объявления обязательные параметры идут перед необязательными. def action(food='ягоды', name): – SyntaxError. При вызове с одним позиционным аргументом он попадёт в name.

```
def make_message(name):
    message = 'Привет, ' + name
    return message
result = make_message('Тоня')
print(result)  # Привет, Тоня
```

Локальные имена: name и message принадлежат вызову функции. print(message) снаружи вызовет NameError, если снаружи такого имени нет.

Как получить данные наружу: вернуть значение через return и сохранить результат вызова. Локальным является имя параметра, а не обязательно само переданное значение.

13 Возвращаемое значение

```
def plus_five(value):
    return value + 5
result = plus_five(2)  # result: 7
print(result)         # 7
print(plus_five(10))  # 15
```

return выражение вычисляет выражение, завершает текущий вызов и передаёт значение в место вызова. Можно вернуть литерал, значение переменной или результат другого вызова.

print показывает данные человеку; **return** отдаёт данные программе. Функция может возвращать результат без печати. Присваивание result = plus_five(2) само ничего не выводит.

```
def show(value):
    print(value)
answer = show(7)  # печатает 7
print(answer)    # None
```

Уточнение: без return или при пустом return функция возвращает None – отсутствие содержательного результата. Код после выполненного return в этом вызове уже не исполняется.

14 Вложенные вызовы и заглушки

```
def plus_five(value):
    return value + 5
def multiply_by_three(value):
    return value * 3
result = plus_five(multiply_by_three(7))
print(result)  # 26
```

`7` → `× 3 → 21` → `+ 5` → `26`

Сначала multiply_by_three(7) возвращает 21; затем plus_five(21) возвращает 26. Значение сохраняется в result и передаётся в print.

```
def later():
    pass  # ничего не делать
def another():
    ...   # значение Ellipsis
```

Пустое тело недопустимо. pass – инструкция-заглушка; ... – отдельное значение Ellipsis, которое тоже подходит как заглушка. Обе эти функции при вызове возвращают None.

15 Программа расчёта объёма

```
PI = 3.14
CM3_TO_LITERS = 0.001
def calculate_volume(height, radius):
    return (PI * height * radius ** 2
            * CM3_TO_LITERS)
first = calculate_volume(9, 5)
second = calculate_volume(15, 7)
third = calculate_volume(20, 9)
print('Первая:', round(first, 2), 'л')
print('Вторая:', round(second, 2), 'л')
print('Третья:', round(third, 2), 'л')
```

Вывод: Первая: 0.71 л; Вторая: 2.31 л; Третья: 5.09 л – каждое сообщение на отдельной строке.

Разбор: константы задают общие коэффициенты; параметры получают размеры; return отдаёт объём; print выводит подпись и результат. Формула написана один раз.

Уточнение к уроку: round(x, 2) округляет до двух знаков после точки; в исходной формуле округление заявлено, но не выполнено. round() не гарантирует два видимых знака: 2.0 не станет 2.00.

Python 3 • Читайте блоки слева направо, затем сверху вниз • Печать A4, масштаб 100%

16 Ошибки и самопроверка

SyntaxError: проверьте кавычки, скобки, двоеточие после def и порядок аргументов.
IndentationError: проверьте отступ тела.

NameError: имя не определено или недоступно здесь.

TypeError: несовместимые типы, недостающий/лишний аргумент, двойное значение параметра.
ValueError: тип аргумента подходит, но значение нельзя обработать, как в int('кот').

Проверьте себя: ① Чем параметр отличается от аргумента? ② Что напечатает print(plus_five(3))? ③ Как передать только food при def action(name='Гость', food='ягоды')? ④ Что вернёт функция с pass?

Ответы: ① Имя в def и значение при вызове. ② 8. ③ action(food='яблоки'). ④ None. Путь данных: аргументы, параметры, вычисление, return, вывод.

Основа: семь приложенных уроков – «Добро пожаловать», «Переменные и их значения», «Типизация и типы данных», «Константы», «Функции», «Параметры и аргументы функций», «Возвращаемое значение».

Дальше по программе курса: ветвления, циклы, коллекции; затем окружение и Git, далее ООП. Эти темы во вложениях анонсированы, но ещё не разобраны.

СВЯЗЬ И НОВЫЕ МАТЕРИАЛЫ

Где искать новые материалы

Новые файлы, а также ссылки на сообщества во ВКонтакте, Telegram, Teletype и Boosty — и, возможно, на YouTube, VK Видео, Rutube, Stepik и Udemy — ищите по этим контактам.

TG
TELEGRAM
t.me/artur_khant

VK
ВКОНТАКТЕ
vk.ru/artur_khant

TT
TELETYPE
teletype.in/@artur_khant

IG
INSTAGRAM
instagram.com/artur_khant

GH
GITHUB
github.com/artur-khant

CV
САЙТ-РЕЗЮМЕ
arturishkildin.cv

Не нашли ссылку на хранилище файлов или нужный цифровой продукт?
Напишите мне в личные сообщения или просто наберите в поиске

АРТУР ХАНТ